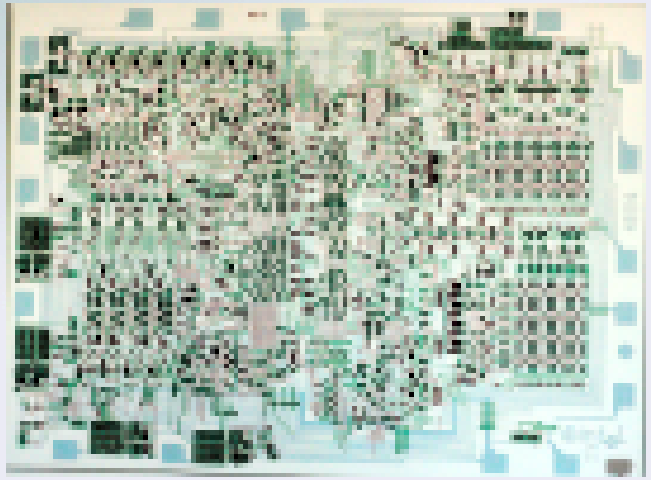

Lecture 5a: Programming: Beginning LC-3 Assembly

CSCI11: **Computer Architecture and Organization**

Chapter 5 in Patt and Patel: Introduction to Computing Systems...

LC 3: Little Computer 3

- 16-bit architecture
- All numbers 2's complement
- Word-based addressing
- 15 opcodes
- bit-numbering right to left

Memory Map

Address	Description
x0000 - x00ff	Trap Vector Table
x0100 - x01ff	Interrupt Vector Table
x0200 - x2fff	OS and Supervisor Stack
x3000 - xfdff	**Available for user**
xfe00 - xffff	Device register addresses

Instruction Set Architecture

- **Operate** - ADD, AND, NOT
- **Data Movement** - LD, LDI, LDR, LEA, ST, STR, STI
- **Control** - BR, JSR/JSRR, JMP, RTI, TRAP
- **Condition Codes** - N - negative, P - positive, Z - zero
- **Addressing**
  - non-memory - immediate, register
  - memory - PC-relative, indirect, base+offset
- **Registers** - 8 General Purpose (R0-R7)

Instruction Set

	15 14 13 12	11 10 9	8 7 6	5	4 3	2 1 0
ADD+	0001	DR	SR1	0	00	SR2
ADD+	0001	DR	SR1	1	imm5	
AND+	0101	DR	SR1	0	00	SR2
AND+	0101	DR	SR1	1	imm5	
BR	0000	n z p	PCoffset9			
JMP	1100	000	BaseR	000000		
JSR	0100	1	PCoffset11			
JSRR	0100	0 00	BaseR	000000		
LD+	0010	DR	PCoffset9			
LDI+	1010	DR	PCoffset9			
LDR+	0110	DR	BaseR	offset6		
LEA+	1110	DR	PCoffset9			
NOT+	1001	DR	SR	111111		
RET	1100	000	111	000000		
RTI	1000	000000000000				
ST	0011	SR	PCoffset9			
STI	1011	SR	PCoffset9			
STR	0111	SR	BaseR	offset6		
TRAP	1111	0000	trapvect8			
reserved	1101					

Figure A.2 Format of the entire LC-3 instruction set. **Note:** + indicates instructions that modify condition codes

LC3 Tools

A collaborative effort to refine an assembler and simulator

- LC3 Tools - Patt and Patel, authors of the textbook and developers of the simulator
- LC3 Tools (UT) - UT/Austin complete overhaul to make multi-platform and refine interface
- **LC3 Tools (GT)** - Georgia Tech complete overhaul of LC3 Tools (UT), to continue to refine along with better debug support. **Be sure to use this version.**

LC3Tools v4.3.1

LC3Tools

hw.asm

```
; hello_world
; Demonstrates:
; how to store a string for output
; how to output a string

; Registers:
; R0 - address of string, required by PUTS
; Memory:
; x3000 - start of program
; x3003 - HW, string to output

    .ORIG x3000
    LEA R0, HW   ; load address of string
    PUTS         ; output string to console
    HALT         ; end program
HW  .STRINGZ "Hello World!"
    .END
```

```
successfully assembled hw.asm into hw.obj
```

LC3Tools v4.3.1

LC3Tools

Registers

Registers	Hex	Decimal	ASCII / Misc
R0	x3003	12291	
R1	x1074	4212	
R2	x944C	-27572	
R3	xC1D3	-15917	
R4	x26D4	9940	
R5	xA995	-22123	
R6	x35F6	13814	
R7	x4853	18515	
PSR	x8002	-32766	CC: Z
PC	x3002	12290	
MCR	x0000	0	

Console

```
Hello World!
```

Debugger

Memory

BP	PC	Address	Hex	Decimal	Label	Instructions
		x3000	xE002	-8190		*LEA R0, HW ; load address of string*
		x3001	xF022	-4062		*PUTS ; output string to console*
	▶	x3002	xF025	-4059		*HALT ; end program*
		x3003	x0048	72	*HW*	*HW .STRINGZ "Hello World!" (H)*
		x3004	x0065	101		*e*
		x3005	x006C	108		*l*
		x3006	x006C	108		*l*
		x3007	x006F	111		*o*
		x3008	x0020	32		
		x3009	x0057	87		*W*
		x300A	x006F	111		*o*
		x300B	x0072	114		*r*
		x300C	x006C	108		*l*
		x300D	x0064	100		*d*
		x300E	x0021	33		*!*
		x300F	x0000	0		
		x3010	x4D1E	19742		
		x3011	x1B01	6913		
		x3012	xF89B	-1893		
		x3013	x09F8	2552		
		x3014	xA6DD	-22819		
		x3015	x2E1D	11805		
		x3016	x3515	13589		
		x3017	x4408	17416		
		x3018	x6835	26677		
		x3019	x9ACB	-25909		
		x301A	x94B3	-27469		

Jump to Location

LC3 Project

Simple Math Calculator (*SMC*)

- Add
- Subtract
- Multiply
- Divide
- Post-fix notation

Post-fix Notation*

Reverse Polish notation (RPN), also known as reverse Łukasiewicz notation, Polish postfix notation or simply postfix notation, is a mathematical notation in which operators follow their operands, in contrast to prefix or Polish notation (PN), in which operators precede their operands. The notation does not need any parentheses as long as each operator has a fixed number of operands.

Prefix Notation: *3 + 4 = 7*

Postfix Notation: *3 4 + 7*

- Wikipedia

Iterative Development

1. Develop math routines using registers
2. Show output via console
3. Request input via console
4. Error checking
5. Extensive stack manipulation

Simple Math Calculator Issues

- 15 *opcodes* or commands, with only **ADD**, **AND** and **NOT** as operators
- *ADD* exists - use for **Add**
- **Subtract**?
- **Multiply**?
- **Divide**?
- **Console In** - Trap instructions, however *ASCII-based*
- **Console Out** - Trap instructions, however *ASCII-based*

LC-3 Simulator

- The *LC-3* Simulator will be our computer for assembly language
- It's value is that it's simplicity enables us to learn quickly
- It is logically complete with AND and NOT
- With ADD, we are able to solve most math equations

However, it requires a significant amount of code to perform very seemingly simple tasks.

I see this as a positive.

How to Begin?

Programming a new project is an iterative task:

1. Start with small tasks, **completed successfully**.
2. **Focus on building confidence** and muscle memory as to how to develop code
3. If something doesn't work, figure out why and repeat until its always right. *Many bugs are caused by sloppy initial work, which manifest more bugs, later.*

Go Slow, to, Go Fast

Demo of LC-3 Simulator *hello_world*

helloworld.asm

```
; helloworld
; Demonstrates:
; how to store a string for output
; how to output a string

; Registers:
; R0 - address of string, required by PUTS
; Memory:
; x3000 - start of program
; x3003 - HW, string to output

    .ORIG x3000
    LEA R0, HW  ; load address of string
    PUTS        ; output string to console
    HALT        ; end program
HW  .STRINGZ "Hello World!"
    .END
```